Cyber Cafe Management System

Comprehensive project report generated from the current Flask implementation

Table of Contents

1. Project Overview

The Cyber Cafe Management System is a Flask-based web application designed to manage computer station usage, simple retail sales, user dashboards, QR-code-based payments, and payment-proof verification. The project is implemented as a lightweight single-application deployment with HTML templates rendered by Flask.

The system currently targets a small cyber cafe environment where a single administrator can manage sessions and payments while customers can view ranks, free stations, and service pricing.

2. Objectives and Scope

- Provide a login system for staff and users
- Allow staff to allocate and release computer stations
- Support a simple POS workflow for services and products
- Introduce QR payment confirmation with screenshot upload
- Allow admin review and approval of payment submissions
- Prepare the project for GitHub hosting and live deployment on a Python platform

3. Feature Summary

- Username/password login for admin and user accounts
- Google OAuth sign-in integration using Google OAuth 2.0 web flow
- Station allocation and live countdown management
- Retail POS cart and checkout flow
- QR-based payment step with screenshot upload
- Admin verification queue for uploaded payment proofs
- User dashboard with ranks, station availability, and pricing

4. System Design

4.1 Architecture

The presentation layer is implemented with server-rendered Jinja templates. The application layer is a single Flask app contained in app.py. The data layer is currently an in-memory collection of Python dictionaries and lists used to store users, stations, retail items, rankings, logs, and payment requests.

Layer	Current Implementation
Presentation	Jinja templates: login.html, admin.html, user.html
Application	Flask routes, business logic, session handling
Storage	In-memory lists/dicts plus uploaded payment screenshots on disk
External Services	Google OAuth endpoints and QR code generation URL

4.2 Core Modules

Module	Purpose
Authentication	Local login and Google OAuth sign-in
Station Management	Allocate, release, and monitor PC sessions
Retail POS	Add items, calculate totals, continue to payment
Payment Verification	Collect screenshot proof and let admin verify or reject
User Dashboard	Display rank, usage summary, pricing, and station availability

5. Authentication and Access Control

The application uses Flask session storage for login state. Local authentication supports static admin and user credentials. Google OAuth is also integrated using the standard authorization code flow, and an optional configured admin email can be elevated to the admin role after Google login.

- login_required protects all logged-in views
- admin_required restricts admin-only routes
- Google sign-in requires GOOGLE_CLIENT_ID and GOOGLE_CLIENT_SECRET
- Redirect URI must match the Google Cloud configuration exactly

6. Payment Flow and Proof Verification

After items are added to the cart, the admin continues to a QR payment page. The total amount, payment reference, receiver information, and QR image are displayed. The payer then uploads a screenshot of the payment confirmation. That screenshot is saved under static/uploads/payments and a payment request is created with pending status.

The admin verification screen lists submitted payments, uploaded screenshots, item details, amount, and status. Each payment can then be verified or rejected.

- Accepted file types: PNG, JPG, JPEG, WEBP
- Status values: pending, verified, rejected
- Payment QR can use a fixed image URL or generated UPI QR data

7. Deployment and Configuration

The application is GitHub-ready and can be deployed to platforms such as Render, Railway, or PythonAnywhere. Runtime configuration is controlled through environment variables for Flask secrets, Google OAuth credentials, and payment settings.

Variable	Purpose
SECRET_KEY	Flask session signing key
FLASK_DEBUG	Enable or disable debug mode
GOOGLE_CLIENT_ID	Google OAuth client identifier
GOOGLE_CLIENT_SECRET	Google OAuth client secret
GOOGLE_ADMIN_EMAIL	Google account that should receive admin access
PAYMENT_UPI_ID	UPI ID used for payment QR creation
PAYMENT_RECEIVER	Display name for payment receiver

8. Limitations and Future Enhancements

- Current data is stored in memory and resets when the server restarts
- Uploaded screenshots are stored on the local filesystem
- No database persistence yet for users, sessions, or payments
- Move all core data to SQLite or PostgreSQL
- Add user-side payment history and notifications
- Add admin analytics dashboards and downloadable reports
- Store uploads in cloud storage for production

Conclusion

The current implementation provides a complete working prototype covering login, station handling, retail checkout, payment proof collection, and admin verification. It is suitable as a strong academic or prototype project and can be extended into a production-ready system with persistent storage and stronger operational controls.